

Terraform State and State file

What is Terraform State?

When we use Terraform to manage infrastructure, Terraform needs to track what it has created, updated, or deleted. This tracking information is stored in a file called the Terraform state file.

- Think of Terraform as a robot that builds infrastructure.
- The **Terraform state** is like its **memory** – it remembers what resources it has created and what their current state is.

Why is Terraform State Needed?

Terraform is declarative, meaning we write code to declare what we want (e.g., an EC2 instance, an S3 bucket). But:

- Terraform doesn't connect to cloud provider every time to see what's already there.
- Instead, it checks the **state file** to understand what it has already done.

What is the Terraform State File?

- It's a JSON file (usually named `terraform.tfstate`).
- It stores metadata and resource mappings.
- It contains all the real-world information about the resources Terraform manages.
- This file is **auto-generated and managed by Terraform**, and we should **not manually edit it**.

Example snippet of a state file:

```
{
  "version": 4,
  "resources": [
    {
      "type": "aws_instance",
      "name": "example",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
```

```
{
  "attributes": {
    "ami": "ami-123456",
    "instance_type": "t2.micro",
    "id": "i-0abcdef123456"
  }
}
]
```

How Terraform Uses the State File

Each time we run commands like:

- **terraform plan**
- **terraform apply**
- **terraform destroy**

Terraform will:

1. Read the current state from **terraform.tfstate**.
2. Compare it with **.tf** code.
3. Query the provider (e.g., AWS, Azure) for the current real-world state (if needed).
4. Figure out the difference (diff).
5. Apply changes to match code and update the state file.



Key Contents of a State File

- Resource Type and Name (e.g., **aws_instance.example**)
- Resource Attributes (e.g., AMI ID, instance type, public IP)
- Dependencies between resources
- Metadata, like module paths and versions



Where is the State File Stored?

By default:

- It's stored locally as **terraform.tfstate** in project directory.

But in team environments:

- It's recommended to store it remotely using a backend (e.g., S3, Azure Blob, GCS, Terraform Cloud) for:
 - **Team collaboration**
 - **State locking**
 - **Versioning**

Example for AWS S3 backend:

```
terraform {  
  
  backend "s3" {  
  
    bucket = "my-terraform-state-bucket"  
  
    key    = "dev/terraform.tfstate"  
  
    region = "us-west-2"  
  
  }  
  
}
```



Terraform State Locking

When using a **remote backend** (e.g., S3 + DynamoDB), Terraform can **lock the state file** during operations to prevent concurrent changes, which avoids **race conditions** and **corrupted states**.



Terraform State Lifecycle

We write **.tf** files.

Run **terraform init** – initializes the working directory.

Run **terraform apply** – Terraform:

- Creates resources
- Writes the result into **terraform.tfstate**

Make changes to **.tf** files.

Run **terraform plan** – Terraform:

- Compares current state file with **.tf** files.
- Shows you what changes will be made.

Run **terraform apply** again – updates the infrastructure and the state file.



Useful Terraform State Commands

terraform show – Shows the contents of the current state.

terraform state list – Lists all resources in the state file.

terraform state show <resource> – Shows detailed info about a specific resource.

terraform state rm <resource> – Removes a resource from state (without deleting it from the provider).

terraform state mv – Moves a resource within the state file (useful during refactors).

Dangers and Best Practices

Danger	Explanation	Solution
State File Contains Sensitive Data	It stores things like passwords, secrets, access tokens.	Use encryption (e.g., S3 encryption, Vault)
Manual Edits Corrupt State	Editing <code>terraform.tfstate</code> manually can break things.	Use <code>terraform state</code> commands instead
Conflicts in Team Environments	Two people changing infra at once can overwrite each other's changes.	Use remote backends with locking
State Drift	Real-world infra might change outside of Terraform	Use <code>terraform refresh</code> or <code>terraform plan</code> to detect drift

Real-world Use Case Example

We define this in [`main.tf`](#):

```
resource "aws_s3_bucket" "my_bucket" {  
  
  bucket = "my-unique-bucket-name"  
  
  acl = "private"  
  
}
```

Run `terraform apply` – it creates the S3 bucket and stores details in `terraform.tfstate`.

Later, you change `acl = "public-read"` and run `terraform apply` again.

- Terraform checks the current state, compares it with your updated code, and updates the bucket's ACL.
- It updates the state file with the new `acl`.

Summary

Concept	Description
Terraform State	A snapshot of your infrastructure Terraform
State File (<code>terraform.tfstate</code>)	JSON file storing resource mappings metadata
Why It Matters	Helps Terraform detect what to change destroy
Storage Options	Local (default) or Remote (recommended for teams)
Contains Sensitive Info	Yes – use secure storage
Don't Edit Manually	Use Terraform CLI commands to manage